Investigating Operating System Variations in IPV6 Implementations

Author: Donovan Rodriguez, dorodriguez@paloaltonetworks.com
Advisor: David Fletcher, david@blackhillsinfosec.com

Accepted: *insert date here* (The date your final draft was uploaded for grading)

Abstract

This is an abstract of a white paper that tells your reader the bottom-line up front. Use the built-in *Abstract* style in this document for proper formatting. Use about one paragraph to sell your paper to a potential reader. This is your opening statement to capture the attention of someone browsing these papers and to get them to read your paper. One approach is to use the first sentence to introduce the problem. Next, spend a sentence explaining why this problem is interesting and worth solving. Move on to other approaches people took to solve this problem and why they work or do not work. Finally, present your solution and why it may be better than other solutions. An abstract should be short and to the point. The abstract should not be repeated in other parts of the paper, such as the introduction. Avoid obvious statements like “This is a research paper."
Another good reference to help you write an abstract:
http://www.ece.cmu.edu/~koopman/essays/abstract.html

1. Introduction to IPv6

Originally introduced as draft RFC 2460 in 1998, IPv6 was designed to be the successor to IPv4 to address address space exhaustion. The key difference to note for IPv6 is its address space, rather than relying on the 32 bit addresses of IPv4, IPv6 is able to take advantage of 128 bits. In the lead up to IPv4 fully being exhausted, some other mechanisms were introduced, such as Network Address Translation in 1994 (The Serial Port, 2023). Ultimately, this led to IPv6 becoming less of an attractive option for the problem and adoption never happened on a wide scale.

Development on the standard still proceeded and was finalized in 2017 with RFC 8200. While the address exhaustion issue was the original catalyst behind IPv6's development. The team also took the chance to incorporate many lessons learned into IPv6 over IPv4. It may comparatively seem like they both were originally proposed such a long time ago, but there was roughly 17 years between IPv4 and RFC 2460 and 36 years between IPv4 and RFC 8200. These improvements take into account all of the problems that were highlighted as deficiencies in IPv4 and would ultimately lead to support from big sources such as ARIN calling out the energy efficiency of IPv6 (Grinius, 2025) due to the fast processing and forwarding times.

1.1. The IPv6 Addresses and header

Before we dive into specific optimizations that were introduced into IPv6 we need to take a look at the format of an IPv6 address and the corresponding header. An IPv6 Address is made up of eight groups of four hexadecimal values, with each value representing 4 bits for a total of 128 bits. Let's start with an example address to make it a bit easier to understand:

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
```

On a first glance, this may seem quite complicated, especially when comparing the address to IPv4 addresses, but there are a few more formatting rules that get applied to

IPv6 addresses that make them easier to understand. The first rule is that leading zeros in each group can be dropped.

```
2001:db8:85a3:0:0:8a2e:370:7334
```

Next, the longest string of consecutive zeros can be compressed and expressed as a double colon (::), leaving us with our final IPv6 address.

```
2001:db8:85a3::8a2e:370:7334
```

This formatted IPv6 address is easier, but is still quite different from IPv4. With some experience, it starts to become much more natural working with these addresses.

Now that we have an understanding of the address format, we can move onto the IPv6 header. Table 1 shows a high level visual representation of all of the different parts of the IPv6 header.

Table 1

Visual Representation of IPv6 Header

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|Version| Traffic Class |           Flow Label                  |
|  (4)  |     (8)       |             (20)                      |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|         Payload Length        |  Next Header  |   Hop Limit   |
|             (16)              |      (8)      |      (8)      |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                                                               |
+                                                               +
|                                                               |
+                     Source Address (128)                      +
|                                                               |
+                                                               +
|                                                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                                                               |
+                                                               +
|                                                               |
+                  Destination Address (128)                    +
|                                                               |
+                                                               +
|                                                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

One key aspect to note is that IPv6 operates using a fixed length header, opposed to the variable length header of IPv4 when using IP Options. To provide a similar functionality to IP options, IPv6 uses the next header field. This field will either point to the higher level protocol (TCP or UDP) or an IPv6 extension header. The extension header has its own next header field, working a daisy chain fashion where the last extension header will point to the higher level protocol. This completely removes the need for the IHL (Internet Header Length) field and the variable size IPv4 header in general.

QoS (Quality of Service) is the next major difference in IPv6. IPv4 solely relied on the ToS (Type of Service or DS (Differentiated Service) field since RFC 2474 published in 1998. The traffic class field in IPv6 is functionally the same as the ToS/DS field, the key difference lies in the addition of the flow label. The flow label is the 5-tuple (Combination of Source IP, Destination IP, Source Port, Destination Port, and Protocol Number) identified by a psuedo-random number chosen by the host to identify the flow. In IPv4, this value was typically calculated by the router, but required the router to look at layer 4 information. This field is intended for QoS purposes, it can be used to identify flows and apply QoS policies, but the most common use case is ECMP (Equal-cost Multi-path) routing. The flow label can also be used by network appliances for firewall decisions or other similar actions that require session identification.

Aside from those fields that are changed or implement new features, the rest of the IPv6 header is mostly kept the same, outside of some changes in name and size to accommodate the address size changes in IPv6. Time-to-live is renamed hop limit in IPv6. The history behind this being a bit interesting, originally it was a time based value in seconds, but as routers got faster, one hop essentially became one second and the value moved over to be hop based, rather than time based. To have this value closer align with the actual meaning, it was renamed in IPv6. Table 2 contains a sample of all of the fields and example values for a real IPv6 header.

Table 2

Example IPv6 Header Values

IPv6 Base Header		
Version	`6 (0x6)`	
Traffic Class	`0xB8`	*DSCP: 46 (EF), ECN: 0*
Flow Label	`0xABCDE`	*QoS flow identification*
Payload Length	`72 (0x0028)`	*Includes ext. header + TCP*
Next Header	`43 (0x2B)`	*Points to a routing extension header*
Hop Limit	`128 0x80`	
Source Address	`2001:db8:cafe:1::10`	
Destination Address	`2001:db8:beef:2::20`	
Routing Extension Header		
Next Header	`6 (TCP)`	*Points to transport layer*
Header Ext. Length	`2 (24 bytes)`	*Length in 8-octet units*
Routing Type	`2`	*Type 2 routing header*
Segments Left	`1`	
TCP Header (Payload)		
Source Port	`443`	*HTTPS*
Destination Port	`52481`	*Ephemeral port*

1.2. IPv6 Optimizations

Some keen readers may have noticed one field that is suspiciously missing from the IPv6 header that is included in IPv4. IPv6 completely omits the checksum from the IPv6 header. The reasoning behind this is that the layer 3 checksum was simply redundant. Layer 2 has a CRC (Cyclic Redundancy Check) that provides data protection for the entire ethernet frame and most layer 4 protocols provide data protection as well. The original specification mandated that UDP checksums were no longer optional. It also began operating off of the assumption that all higher level protocols would provide checksums. This wasn't a problem for most applications that ran over TCP or UDP since the changes to UDP now required checksums, but

some network protocols that were transported directly over the network without layer 4 needed to be rewritten to make up for this.

Aside from the checksum being redundant, the removal of the checksum actually provided a key optimization for large organizations. Every time a packet passes through a router and the TTL is decremented in IPv4, the checksum would need to be recalculated since the header was changed. The performance hit for most users is negligible, but when you're a massive organization that is serving millions of users, the impact is much more significant, providing a key performance optimization for many larger organizations.

ICMPv6 was written to introduce NDP (Neighbor Discovery Protocol). Described in RFC 4861, NDP is a set of ICMPv6 message types that replace ARP and other IPv4 neighbor and router discovery mechanisms. NDP operates using a series of multicast messages to discover and link layer 2 addresses to IPv6 addresses. The functionality of NDP directly led to ARP being completely deprecated from IPv6 with NDP relying on much more efficient multicast and link-local addressing methods, rather than ethernet broadcasts.

The use of multicast in IPv6 in general is another great optimization in IPv6. Broadcast messages are completely removed from IPv6. The primary issue with broadcasts was that by their nature, every host on the segment had to process the packet up through the network stack before discarding it, wasting CPU cycles on devices that had no use for the traffic. To achieve similar functionality, IPv6 introduces a set of well-known multicast addresses that are scoped to limit which hosts must process a given message. For example, NDP's solicited-node multicast address (ff02::1:ff00:0/104) targets only nodes whose IPv6 address shares the same last 24 bits, meaning a neighbor solicitation interrupts one or a small number of hosts rather than every device on the subnet as an ARP broadcast would in IPv4.

The lessons learned from years of IPv4 development allowed for other optimizations that were in IPv6 as well, the examples I discussed were introduced to give you an understanding about the benefits of using IPv6 outside of simply solving the IPv4 address exhaustion issue. It's been well over 30 years since IPv6 was

introduced and still hasn't replaced IPv4, I personally think the performance optimizations are well worth it and the problem why most network engineers and admins have not swapped over is a lack of understanding behind IPv6.

1.3. IPv6 Implementation Across Operating Systems

Part of advocating for the widespread adoption of IPv6 is helping to demystify the protocol and address the practical concerns that keep organizations on IPv4. Many existing network tools and standard operating procedures were built around IPv4 assumptions and may not function the same way under IPv6. OS fingerprinting technologies, for example, rely on IPv4-specific header behaviors to differentiate between operating systems, and those distinguishing characteristics may look entirely different once traffic moves to IPv6. My goal with this research is to investigate how IPv6 is actually implemented across different operating systems in an effort to help close that knowledge gap and reduce some of the uncertainty that slows adoption. What exactly are the differences between IPv6 implementations across Windows, Linux, BSD, and macOS? How does each operating system handle malformed IPv6 packets? How do they respond to standard requests? By testing each of these platforms and documenting the results, I hope to provide a practical reference that can help network engineers and security professionals better understand what to expect when they make the transition.

2. Research Method

The approach to this research is built around controlled experimentation in an isolated lab environment. The goal is to systematically compare how different operating systems implement IPv6 protocols by sending specially crafted traffic to each platform and observing the responses. Rather than examining source code or relying on vendor documentation, this study takes a black-box approach, treating each operating system as an independent subject and measuring its observable behavior against the standards defined in the relevant RFCs. This method was chosen because it mirrors the perspective of a network engineer or security

professional operating in a production environment, where the internal workings of a given OS are not visible, but the network behavior is.

2.1. Test Environment

The testing environment consists of an isolated IPv6 lab network running on VMware Workstation. The network will be segmented by VMWare into an isolated IPv6 subnet. The target hosts include Windows 11, Fedora 43, macOS Tahoe, and FreeBSD 15.0, each representing a major OS family commonly found in enterprise environments. Traffic capture is performed on the physical host machine by running Wireshark and tcpdump against the virtual NIC of the VM network, and custom Python scripts leveraging the Scapy library are used to craft packets targeting the different testing scenarios.

The decision to use a virtualized environment was made for practical reasons. VMs allow for rapid deployment, snapshotting for reproducibility, and the ability to test multiple OS versions without requiring a large physical hardware inventory.

2.2. Testing Methodology

The tests are organized into three broad categories that together cover the major areas of IPv6 protocol implementation: malformed packet handling, supporting protocol behaviors, and core protocol mechanics. Each category targets a different aspect of how operating systems interact with IPv6 traffic, and each is designed to reveal behavioral differences that carry security implications.

The first category, malformed packet handling, examines how each operating system responds to traffic that deviates from the standards. This includes packets with invalid header fields, fragmentation anomalies, corrupted checksums, and the use of reserved or deprecated protocol values. The measured variables in this category are whether the OS accepts or rejects the packet, how strictly it validates the packet against the RFC specifications, and whether it generates an error message in response.

The second category focuses on supporting protocols, specifically Neighbor Discovery Protocol (NDP), Router Advertisements, address autoconfiguration via SLAAC and DHCPv6, and ICMPv6 message processing. For these tests, protocol messages are generated with various parameter combinations to observe how each target OS handles neighbor cache management, router selection, address generation, and error responses.

The third category addresses core IPv6 protocol mechanics, including extension header handling, flow label behavior, hop limit processing, and address type support. The tests in this category examine how each OS parses and processes extension headers in various orderings and configurations, how it generates and assigns flow labels, and whether it supports the full range of IPv6 features defined in the specification.

For each individual test, the procedure follows a consistent pattern. Test traffic is crafted using Scapy, sent to the target VM, and the response (or lack of response) is captured on the physical host against the VM's virtual NIC using Wireshark and tcpdump. Each test is run against every target operating system under the same network conditions to ensure that any observed differences are attributable to the OS implementation rather than environmental factors.

2.3. Data Collection and Analysis

Data collection relies primarily on packet captures taken on the physical host machine against each target VM's virtual NIC. Each test generates a capture file that records the exact packets sent and the corresponding responses from the target OS. These captures serve as the raw dataset for both quantitative and qualitative analysis.

On the quantitative side, the analysis includes acceptance and rejection rates for malformed packets, flow label entropy measurements, and extension header support coverage across platforms. On the qualitative side, each observed behavior is categorized as RFC-compliant, implementation-specific, or divergent. The combination of these approaches produces behavioral comparison matrices that

map each operating system's response across the full range of tests, making it straightforward to identify where platforms diverge.

3. Findings and Discussion (Exposition of the Data)

3.1. Sub-Sections of the Findings and Discussion Section

3.2. Additional Technical Data to Format the Paper

3.2.1 Labeling figures

3.2.2. Special formatting of code and file names

3.2.3. Using color in your paper

4. Recommendations and Implications

4.1. Recommendations for Practice

4.2. Implications for Future Research

5. Conclusion

References

The Serial Port. (2023, June 2). THE UNTOLD STORY: How the PIX Firewall and NAT Saved the Internet. YouTube. https://www.youtube.com/watch?v=GLrfqtf4txw

Deering, S. E., & Hinden, B. (1998, December 1). *Internet Protocol, Version 6 (IPv6) Specification*. IETF. https://www.rfc-editor.org/rfc/rfc2460.html

Deering, S. E., & Hinden, B. (2017, July 1). Internet Protocol, Version 6 (IPv6) Specification. IETF. https://www.rfc-editor.org/rfc/rfc8200.html

Narten, T., Nordmark, E., Simpson, W., & Soliman, H. (2007). Neighbor Discovery for IP version 6 (IPv6). Www.rfc-Editor.org. https://www.rfc-editor.org/rfc/rfc4861.html

Nichols, K., Blake, S., Baker, F., & Black, D. (1998). Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers. https://www.rfc-editor.org/rfc/rfc2474.html

Vincentas Grinius. (2025, December 30). What Is More Energy-Efficient: IPv4 or IPv6? Arin.net. https://www.arin.net/blog/2025/12/30/energy-efficiency-ipv4-ipv6-ipxo/

Appendix
Common Style Mistakes

A few common style mistakes:

- Avoid phrases like “The author believes…”.
- Do not introduce each section with a sentence like “This section will cover…”
- Graduate writing involves complex and compound sentences, but you must guard against confusing the reader. Essentially, we do not want choppy-sounding paragraphs and sentences.
- If the paper still makes sense, if you remove a sentence or a word, remove it.
- Write a “story”. Avoid bulleted lists (like this one) unless they are necessary. Rule of thumb: 2-3 per paper are ok. Check the Purdue Owl for APA style seriation rules (list-making rules – this can be done in sentences rather than bulleted lists if short)
- Vary your sentence structure. Avoid starting two consecutive sentences with the same word. Never end a sentence and begin the next sentence with the same word.